

File Guide: How the Supersession Experiment Works

What every file in `experiment/` does, in the order it is used, with a real OIR bulletin followed through each step.

Project: `oir-hub-ai` · Folder: `experiment/`

Research question: when a new OIR announcement arrives, can a small local LLM (Qwen3-4B) decide **UPDATE / NEW / REDUNDANT** better than simple rules?

Written 30 September 2026.

Contents

1. The problem in one story
2. The whole pipeline at a glance
3. The running example
4. The files, one by one (10 files)
5. Where the data lives
6. Key concepts explained simply
7. What to do next

1. The problem in one story

Imagine you run a notice board for international students. Last year you pinned "**2025 Japan scholarship, deadline March 1.**" Today a new paper arrives: "**2026 Japan scholarship, deadline April 10.**" What should you do with it?

Label	Meaning	What the system should do
UPDATE	The new paper replaces an older one (same program, something changed)	Add the new one, take the old one down
NEW	It is about something the board does not have yet	Add it
REDUNDANT	It is the same paper again (a re-post)	Skip it

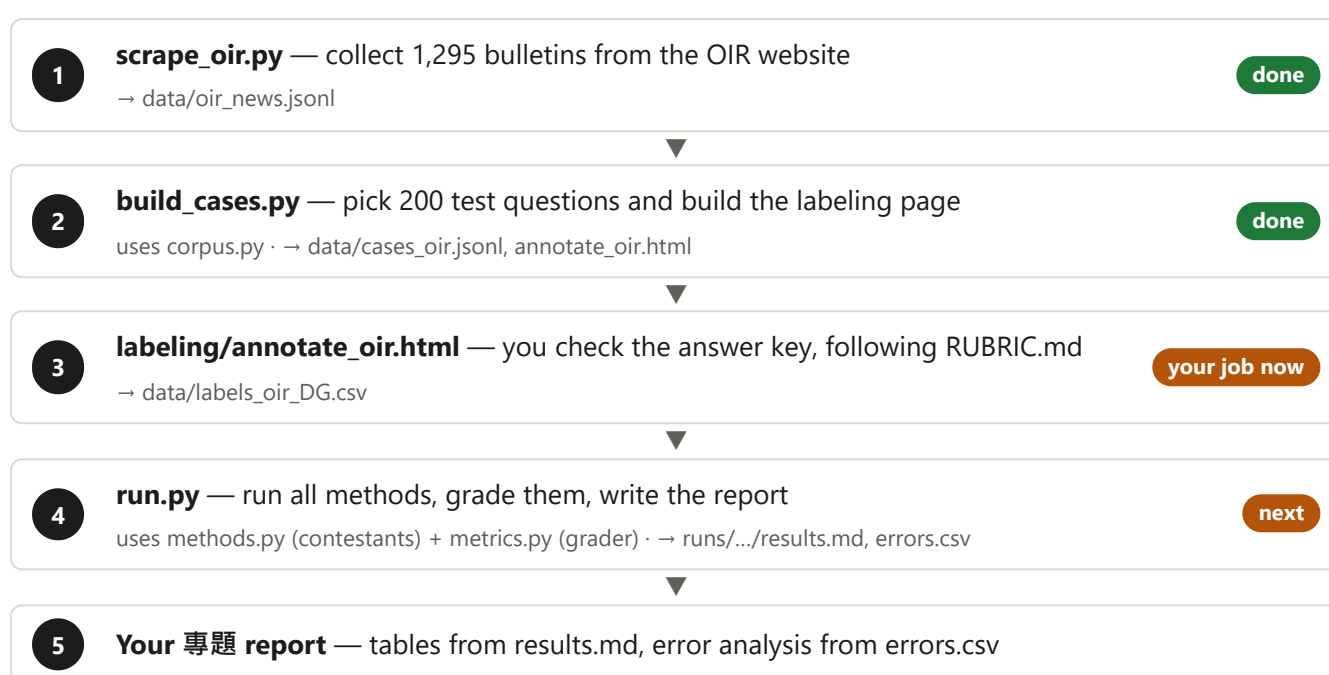
The OIR chatbot has the same problem: its "notice board" is its document database. If an old bulletin is not taken down, students read wrong deadlines.

The research question. Who makes this decision better: **simple rules** (for example "same title means same program") or an **AI model** that reads both bulletins? The experiment is a fair contest between them, and "the rules are good enough" is also a valid result.

A fair contest works like grading an exam:

1. **Questions:** 200 real OIR bulletins, each asking "UPDATE, NEW or REDUNDANT?"
2. **Answer key:** the correct answer for each question, checked by a human (you).
3. **Students:** the rules (B0–B4) and the AI setups (M1–M4) answer every question.
4. **Grading:** the program compares their answers with the answer key.
5. **Result:** who scored higher, and why the others made mistakes.

2. The whole pipeline at a glance



Kind of file	Files	Do you run it?
run once	scrape_oir.py, build_cases.py	Already done
you use	annotate_oir.html, run.py	Yes — these are your two steps
helper	corpus.py, methods.py, metrics.py	Never directly; other files use them
read	RUBRIC.md, PLAN.md, README.md	Read them
check	test_metrics.py	Optional: proves the grader is correct

3. The running example

This guide follows one real bulletin through every file:

oir-944 · 2023-10-27

公告 || 2024年度日本國際交流基金會(The Japan Foundation)經費資助計畫

"日本國際交流基金會經費資助計畫是以公開招募方式... 詳細申請資訊與時程如附件檔案。"

One year earlier, OIR posted the 2023 edition as **oir-793** (2022-10-14). So the correct answer is **UPDATE**, replacing **oir-793**: same program, new year.

4. The files, one by one

① `scrape_oir.py` — collect the bulletins done

Input The public OIR website, `https://oir.thu.edu.tw/redirect.php?ID=news&Sn=1 ... Sn=1350`

Output `data/oir_news.jsonl` (1,295 bulletins) and a backup of every page in `data/raw/`

Run it? No need. It is finished. Re-running is fast because it reuses the backups.

What "scraping" means: a program reads web pages automatically instead of a person clicking through them. Every page it reads is public.

What it does for each number from 1 to 1,350:

1. If `data/raw/944.html` already exists, read it from disk (no download).
2. Otherwise download the page and save it to `data/raw/`.
3. Find the useful parts by their markers in the page code: the title (`<h3 class="page-title">`), the date (after the calendar icon), the body (`<div class="page-content">`).
4. Strip the HTML tags so only plain text remains.
5. Wait 1 second before the next page, so the university server is not overloaded.

It takes the bulletin text, not the attachments. The PDF or Word files linked from a bulletin are not downloaded; only their links are saved. Some bulletins just say "see attachment", which makes them hard to judge. Those cases are marked `unsure`.

```
{"doc_id": "oir-944", "sn": 944, "date": "2023-10-27",  
  "title": "公告||2024年度日本國際交流基金會(The Japan Foundation)經費資助計畫",  
  "text": "日本國際交流基金會經費資助計畫是以公開招募方式...", "links": [...], "url": "..."}  
...
```

One line of `oir_news.jsonl`. ".jsonl" = one JSON record per line.

② corpus.py — the toolbox helper

Used by build_cases.py, methods.py, run.py

Run it? Never directly.

It gives the other files four tools:

Tool	What it does
Load (<code>load</code>)	Reads the bulletins, drops the 131 undated pages (<code>0000-00-00</code> , school profiles with no date), sorts by date.
Knowledge base (<code>kb_indices</code>)	For a bulletin, returns every bulletin dated <i>strictly before</i> it: what the chatbot knew on that day. You cannot replace a bulletin from the future.
Clean titles (<code>norm_title</code>)	Removes years, seasons, prefixes and numbers so editions of the same program compare equal.
Similarity (<code>cosine_scores</code> , <code>bm25_scores</code>)	Two ways to measure how alike two bulletins are: by meaning (e5) and by shared words (BM25).

"公告||2024年度日本國際交流基金會(The Japan Foundation)經費資助計畫"

↓ `norm_title`

"日本國際交流基金會thejapanfoundation經費資助計畫"

← the 2023 edition gives the same text

e5 (meaning): a model turns each bulletin into 768 numbers (a *vector*). Similar meaning gives similar vectors; *cosine similarity* compares them (1.0 = same meaning). **Finding:** all OIR bulletins are about exchanges and scholarships, so they all score about 0.92–0.95 against each other, and the score hardly separates "same program" from "different program".

BM25 (words): a classic search-engine score. It counts shared words and gives rare words (「基金會」) more weight than common ones (「公告」). Chinese has no spaces, so text is split into overlapping character pairs: 日本國際 → 日本, 本國, 國際.

③ build_cases.py — make the test questions done

Input `data/oir_news.jsonl`

Output `data/cases_oir.jsonl` (200 questions) and `labeling/annotate_oir.html` (labeling page)

Run it? No need. It is finished.

Why 200 and not all 1,295? Every answer must be checked by hand.

Why not random? About 9 in 10 bulletins are NEW, so a random 200 would have very few UPDATES. It picks from three *pools*:

Pool	How chosen	Count	Purpose
title	An older bulletin has a very similar cleaned title	90	Recurring programs and look-alike traps
semantic	No title match, but meaning is very close	40	Reworded successors that title rules miss
random	Truly random (2015 or later)	70	Keeps the natural mix realistic

Candidates: for each question it gathers ~10 older bulletins that might be the replaced one (top 5 by meaning, top 5 by words, top 5 by title), shown sorted by date so no method is favoured.

Fairness: the 200 questions are shuffled so you cannot see the pool, and 40 are set aside for a classmate to label independently (the κ subset).

```
{"case_id": "oir-180", "doc_id": "oir-944", "date": "2023-10-27", "pool": "title",  
  "candidates": ["oir-793", "oir-752", "oir-577", ...]}
```

④ RUBRIC.md — the rules for the correct answer [read](#)

Two people will disagree about "is this an UPDATE?" unless the rules are written down. The core procedure:

1. **Same program?** Is there an older bulletin about the same program, scholarship or event? If not → **NEW**.
2. **Anything actionable changed?** Deadline, year, fee, eligibility, procedure? Yes → **UPDATE**. No → **REDUNDANT**.

Tricky case	Answer
2025 edition → 2026 edition, even with identical wording	UPDATE
Two different scholarships with the same layout ("template trap")	NEW
Deadline extension (「延長」) or correction (「更正」)	UPDATE
Results / admission list (「錄取名單」「放榜」)	NEW
English and Chinese versions of the same announcement	REDUNDANT
Second selection round (「第二次甄選」) of the same call	UPDATE
Numbered lecture in a series (「系列之二十八」)	NEW

Example: oir-944 is the same Japan Foundation program in a new year → **UPDATE**, target oir-793.

⑤ labeling/annotate_oir.html — the answer-key editor [your job now](#)

Input The draft answers `data/labels_oir_draft.csv` (loaded with Import CSV)

Output Your checked answers, saved with Export CSV as `data/labels_oir_DG.csv`

Built from `labeling/annotate_template.html` (the empty design; do not edit it)

It opens in the browser (double-click), works offline, and autosaves in that browser.

- **Left:** the question bulletin. **Right:** ~10 older candidates.
- Keys: `u` UPDATE · `n` NEW · `r` REDUNDANT · `1-9` tick the replaced bulletin · `j` / `k` next/previous.
- Not listed? Use the search box to find any older bulletin by title.
- Write a **one-sentence reason** in notes. The AI method M3 learns from these reasons.

```
case_id,gold_label,gold_target,changed_field,notes,annotator,seconds
oir-180,UPDATE,oir-793,cycle,2024 Japan Foundation grant programs replace the 2023
edition,DG,40
```

Important: the draft answers were written by an AI (Claude). Because the experiment tests an AI, the answer key must be checked by a human, or the results would only measure agreement with Claude. Start with the 17 cases flagged `unsure`. Your classmate must label the 40 κ cases *without* seeing the draft.

⑥ methods.py — the contestants helper

Every method answers each question with a label and, for UPDATE/REDUNDANT, which older bulletin it means.

Method	How it decides
B0 majority	Always gives the most common answer. The "no thinking" floor.
B1 metadata	Identical text → REDUNDANT. Same cleaned title → UPDATE (or REDUNDANT if the text is nearly identical). No match → NEW.
B2 cosine (e5)	Takes the most similar older bulletin by meaning. Score above a high cut-off → REDUNDANT, above a lower one → UPDATE, else NEW.
B3 BM25	Same idea with the word-overlap score.
B4 reranker	A stronger model reads the two texts together and scores them 0–1; same cut-off idea.
M1 AI alone	Qwen sees only the new bulletin and guesses. Cannot name a target.
M2 AI + candidates	Qwen sees the new bulletin plus the 5 most similar older ones and answers in fixed JSON.
M3 AI + examples	Like M2, plus 3 solved examples (one per label) taken from your dev-set reasons.
M4 cascade	Cheap rules first (identical text → REDUNDANT; very low similarity → NEW); asks Qwen only when unsure. Your own method contribution.

Cut-offs ("thresholds"): nobody knows the best cut-off in advance, so the program tries many values on the **dev** questions, keeps the best, and then uses it unchanged on the **test** questions.

What the AI sees and answers:

```
[rules]  UPDATE = same program, something a student acts on changed ...
INCOMING ANNOUNCEMENT  title / date / text of oir-944
EXISTING DOCUMENTS      [C1] oir-793 ... [C2] ... [C5]

answer → {"rationale": "Same Japan Foundation grant program, new 2024 cycle.",
          "label": "UPDATE", "target": "C1", "confidence": 0.9}
```

Two bugs fixed during testing (worth a sentence in your report): when `target` was optional the AI said UPDATE without naming a bulletin, so it is now required. Examples with a placeholder reason ("This is UPDATE.") made the AI stop reasoning (M3 fell to 0.21), and real reasons fixed it (0.65).

⑦ `metrics.py` — the grader helper and ⑧ `test_metrics.py` — checks the grader

`metrics.py` turns answers into scores and tells you whether a difference is real. `test_metrics.py` checks every formula on tiny examples with known answers; it should print `ok` for each test.

Measure	Plain meaning
Precision	When the method says UPDATE, how often is it right?
Recall	Of the real UPDATES, how many did it find?
F1	One number combining precision and recall.
Macro-F1 (main score)	Average F1 over UPDATE, NEW and REDUNDANT, each counting equally. "Always UPDATE" gets 68% accuracy but only 0.27 macro-F1.
Target accuracy	For real UPDATES: did it name the right old bulletin?
Confusion matrix	A table of which answer was confused with which.
Bootstrap CI	Re-draw the test set 10,000 times; the middle 95% range shows how much the score could move by chance.
McNemar test	Looks only at questions where one method was right and the other wrong. 15 vs 3 → real difference; 8 vs 7 → a tie.
Holm correction	Makes the bar stricter when several tests are run at once.
Cohen's κ	Agreement between you and your classmate beyond chance. ≥ 0.6 → the rubric is clear enough.

Worked example: 80 real UPDATES; the method said UPDATE 90 times, 72 correctly → precision $72/90 = 0.80$, recall $72/80 = 0.90$, $F1 \approx 0.85$.

⑨ run.py — run the contest next

Input Your answer key `data/labels_oir_DG.csv`

Output A new folder `runs/oir_<date_time>/` with `results.md`, `errors.csv`, `predictions.csv`

```
.venv/Scripts/python experiment/run.py oir experiment/data/labels_oir_DG.csv --no-llm #  
rules only, seconds  
.venv/Scripts/python experiment/run.py oir experiment/data/labels_oir_DG.csv #  
everything, ~1 hour
```

1. **Load** your 200 answers and the 200 questions.
2. **Split** them: 40% **dev** (practice), 60% **test** (the real exam), keeping the label mix equal. Fixed seed, so the split is the same every run.
3. **Compute signals** (cosine, BM25, title, reranker) and cache them in `data/cache/`.
4. **Tune** each method on dev only.
5. **Test** each method on test; AI answers are cached, so re-running costs nothing.
6. **Grade** with `metrics.py` and **write the report**.

Output file	Use it for
<code>results.md</code>	All tables for the report: scores, significance, confusion matrices, errors per method.
<code>errors.csv</code>	Every wrong answer with the AI's reasoning. Your error analysis comes from here.
<code>predictions.csv</code>	Every answer from every method.

The one rule you must never break: tune only on dev, report only on test. Tuning on test makes the numbers meaningless.

⑩ PLAN.md and README.md — documentation read

`PLAN.md` is the research plan: question, hypotheses, data, methods, metrics, timeline and limitations.

`README.md` lists the commands to run.

5. Where the data lives

Location	Contents
experiment/data/raw/	Backup of every downloaded web page (1.html , 2.html , ...)
experiment/data/oir_news.jsonl	Main data: all 1,295 bulletins with title, date, text
experiment/data/cases_oir.jsonl	The 200 test questions and their candidates
experiment/data/labels_oir_draft.csv	Draft answers written by Claude (to be checked)
experiment/data/labels_oir_DG.csv	Your checked answer key (created when you export)
experiment/data/cache/	Saved calculations and AI answers, for speed. Safe to ignore
experiment/runs/	One folder per run, with the results
data/*.pdf (project root)	The brochure PDFs, used only as a small pilot set for testing the code

6. Key concepts explained simply

Term	Meaning
Bulletin	One announcement post on the OIR news page.
Knowledge base (KB)	All bulletins older than the one being judged.
Case / question	One bulletin plus its knowledge base, to be labeled UPDATE / NEW / REDUNDANT.
Target	The older bulletin being replaced (UPDATE) or duplicated (REDUNDANT).
Gold label / answer key	The correct answer, decided by a human.
Baseline	A simple method the AI must beat to be worth using.
Dev / test split	Practice questions for tuning, and exam questions for reporting.
Threshold	A cut-off number, such as "UPDATE if the similarity is above 0.94".
Embedding / vector	A list of numbers representing a text's meaning.
Template trap	Two different programs whose bulletins look alike, so similarity methods wrongly call it UPDATE.
Reworded successor	A new edition written differently, so title rules miss the UPDATE.
Cascade	Use cheap rules when confident, the AI only when unsure.
Data leakage	Letting test information influence training or tuning, which makes scores falsely high.

7. What to do next

1

Check the answer key. Open `labeling/annotate_oir.html` → Import CSV → `labels_oir_draft.csv` → fix wrong answers (start with the 17 `unsure`) → Export CSV as `labels_oir_DG.csv`.

now



2

Second annotator. Send `labeling/annotate_oir.html` and `RUBRIC.md` (not the draft) to a classmate; they tick "k subset only" and label 40 cases.



3

Run the baselines with `--no-llm`, then the full experiment.



4

Error analysis. Open `errors.csv`, pick wrong answers, and explain why each method failed.

Self-test: if you can explain, without notes, why cosine similarity fails on look-alike programs and why you must never tune on the test split, you understand the core of this project.